Audio Deep Learning

Assignment #4

Text To Speech Synthesis

Assignment overview and motivation

- *Format: Individual or group (max 3 students) with offline defense*
- *Grading: 17 points*
- *Timeline: Assignment # 4 available on 29th June, Code submission deadline till 15th July, and offline defense deadline till 23th July*
- ***Submission via Google Form**: https://forms.gle/czeSr99PZYDv3j327*

While completing this assignment, you will develop and strengthen the following skills:

- *Approaching a real-world **Text-to-Speech (TTS)** synthesis problem by transforming text into natural-sounding speech while considering speaker identity, pronunciation accuracy, and speech naturalness.*
- *Conducting effective **Exploratory Data Analysis (EDA)** on speech corpora, transcripts, and acoustic characteristics (e.g., utterance duration, phoneme distributions, speaker statistics, pitch, speaking rate, and recording quality) to better understand the dataset and its challenges.*
- *Designing a reliable **speaker-independent** training and validation strategy that prevents speaker leakage between dataset splits while ensuring robust estimation of model generalization to unseen speakers.*
- *Utilizing modern **neural Text-to-Speech architectures** (e.g., Tacotron2, VALL-E, xTTS, CosyVoice, Parler-TTS, or Flow Matching-based models) together with pretrained speech representations, neural codecs, or discrete acoustic tokens to generate high-quality speech.*
- *Performing a comprehensive **ablation study** on model architectures, tokenization or codec representations, conditioning strategies (speaker embeddings, prompts, or style descriptions), vocoders, training configurations, and inference techniques to identify the most effective TTS synthesis pipeline.*

Task

Task Selection:

- *Perform TTS Task on LJ Speech*

EDA and Metric Analysis:

- *Perform Exploratory Data Analysis (EDA)*

- *Proposed Target Metrics from XTTS V2 : CER; UTMOS; SECS (You can find links to implementation in the paper). Analyze them*
- *Suggest complementary metrics to capture other aspects of performance.*
- *Propose an alternative target metric, with justification.*

Validation:

- *Split the original dataset into Train/Test/Validation splits*
- *Find open-source text dataset (not big) for additional Testing*
- *Generate some edge-case texts with an LLM for one more evaluation. Justify selected edge cases*
- *Discuss which metrics are applicable and not to generate (only text datasets)*

GPT TTS:

- *Finalize code in Module 5 Lecture 2 notebook for GPT TTS*
- *Train using Pytorch or/and Pytorch-Lightning*
- *Inference using different sampling strategies: greedy decoding, top-k sampling, top-p/nucleus sampling, temperature scaling, repetition penalty, beam search and others*
- *Collect all relevant results:*
    - *metrics CER; UTMOS; SECS;*
    - *inference examples for different test sets.*
- *Prepare an ablation study on the tested models and inference configurations.*
- *[Bonus] Repeat experiments but with Lamma instead of GPT*

RLHF approach:

- *Apply an RLHF technique (e.g., DPO) to optimize a non-differentiable metric. Check this paper . Suggestion: Apply to the tune model from the previous point*
- *Collect all relevant results:*
    - *metrics CER; UTMOS; SECS;*
    - *inference examples for different test sets.*
- *Prepare an ablation study on the tuned model compared with the baseline*

Final Report

- *Prepare a comprehensive report including:*
    - *data insights,*
    - *feature engineering,*
    - *tried algorithms and models,*
    - *chosen hyperparameters,*
    - *metrics,*
    - *insights and conclusions.*
- *Reports may be prepared in LaTeX or Markdown (README.md).*

NOTES

- *Do not hesitate to include failed ideas or experiments in your report. Documenting unsuccessful attempts is a strong indication that you systematically explored different approaches and gradually arrived at the best-performing solution, rather than relying on a "lucky shot." Moreover, analyzing failed approaches can often lead to valuable insights about the dataset, the task itself, and the behavior of deep learning algorithms. Such reflections may also highlight potential directions for future improvements and demonstrate a deeper understanding of the problem-solving process.*

- *Keep your EDA concise. Avoid overloading it with excessive plots, histograms, or tables. Focus only on key aspects of the data (e.g., missing values, target distribution). If a part of your EDA does not lead to meaningful conclusions, you likely don't need it (for example, full pairwise correlation matrices). Do NOT follow the typical Kaggle-style EDA — most of them are far too verbose and unfocused.*
- *Prepare your presentation carefully. A messy speech with too much code or too many files makes it nearly impossible for the audience to follow. Think in advance about what you want to highlight*

Grading Policy and Defense

Grading Policy is outlined in the table

Criterion	***Points***
Overall correctness and completeness of the submitted code for the lab assignment, overall quality (metrics) of the proposed solution	*5*
Practical question about the submitted code during defense	*7*
Theoretical part during defense (2 questions + 1 exercise)	*5*
Criterion	***Points***

The defense will be conducted during a practical session, where the Lecturer will evaluate your code, report, and presentation skills. It is important to emphasize that the assessment will not be limited to metrics, analysis, and the performance of your models, but will also take into account the overall quality of your code, including readability, structure, and adherence to good practices. During the defense, the Lecturer will evaluate each student's individual contribution, demonstrated skills, and depth of knowledge. In addition, up to 2 theoretical questions may be asked; these will be based on the questions outlined in Appendix 1, but will not necessarily be restricted to them.

Exercise with practical tasks similar to Appendix 2 will be evaluated in the form of a written test on one of the practical sessions

AI Tools usage policy

Scope

This policy applies to all generative AI tools that produce or modify text, code, images, or other digital content. This includes, but is not limited to:

- *Text generation tools: ChatGPT, Claude, Gemini, Mistral, LLaMA-based models, etc.*
- *Code generation tools: GitHub Copilot, OpenAI Codex, TabNine, etc.*
- *Language assistance tools: Grammarly, DeepL, Google Translate, etc.*
- *Local LLMs or self-hosted AI models.*

Allowed Uses

You may use AI tools for the following purposes, provided they comply with the conditions below:

- *Spell Check & Style Correction. Example: Using Grammarly to correct grammar and improve style in a lab report or paper review.*
- *Translation. Example: Using DeepL or Google Translate to translate sections of a document into another language.*
- *Idea Generation & Pitching. Example: Brainstorming possible approaches for solving a problem or generating research ideas.*
- *Self-Education. Example: Using ChatGPT to explain an unfamiliar algorithm or concept.*
- *Library & Documentation Exploration. Example: Asking AI to summarize PyTorch documentation for a specific class or function.*
- *Code Assistance (Partial). Writing or correcting specific functions or methods (not entire solutions). Examples: Write a function that reads data and converts it to another format. Implement a data collator with padding to the longest sequence. Create an image augmentation function for training.*

Conditions for Use

Full Transparency:

- *Any use of AI tools must be disclosed in your submission.*
- *Include all prompts, responses, and/or screenshots showing your interaction with the AI tool.*
- *If using online tools, include chat links (if available) or full transcripts in an appendix of your report.*

Partial Contribution Only:

- *AI tools should support your work — they must not produce the complete assignment, lab, or report.*
- *You must be able to explain and defend any AI-generated content during oral defense or Q&A*

Examples

Acceptable Uses:

- *"Correct grammar and style in this paragraph of my report."*
- *"Write a function to load images from a directory and resize them."*
- *"Suggest 3 ideas for improving model generalization."*

Unacceptable Uses:

- *"Write the entire peer review for this paper."*
- *"Generate the complete solution code for this assignment task."*

Appendix 1 (Theoretical Questions)

- *Explain the difference between **Text-to-Speech (TTS)** and **Voice Conversion (VC)**. What information is provided as input to each task, and what aspects of speech must each system generate or preserve?*
- *What are the primary challenges in building high-quality TTS systems? Why are naturalness, intelligibility, pronunciation accuracy, prosody, and speaker similarity often competing objectives?*
- *Explain the typical TTS pipeline. What are the roles of text normalization, grapheme-to-phoneme conversion, acoustic modeling, and vocoding?*
- *Why is text normalization important for TTS? Give examples of text that cannot simply be read character by character.*
- *Explain the difference between **grapheme-based** and **phoneme-based** TTS systems. What are the advantages and disadvantages of each approach?*
- *What is a Mel-spectrogram, and why is it commonly used as an intermediate representation in neural TTS systems?*
- *Explain the sequence-to-sequence paradigm used in **Tacotron2**. What are the responsibilities of the encoder, attention module, and decoder?*
- *Why is the attention mechanism crucial in Tacotron2? What types of synthesis errors can occur when attention fails?*
- *Explain the role of teacher forcing during Tacotron2 training. Why is inference more difficult than training?*
- *Why do autoregressive TTS models generally produce higher latency than non-autoregressive models?*
- *Explain the motivation behind **Vector Quantized Variational Autoencoders (VQ-VAE)**. Why is discrete latent representation useful for modern TTS?*
- *Describe the codebook quantization process in VQ-VAE. What are the purposes of the commitment loss and the codebook loss?*
- *Explain how **Tortoise TTS** combines autoregressive generation and CLVP reranking. Why does this improve speech quality?*
- *What is **CLVP (Contrastive Language-Voice Pretraining)**? How is it conceptually similar to CLIP?*
- *Explain the core idea behind **VALL-E**. Why can speech generation be formulated as a language modeling problem over discrete acoustic tokens?*
- *What are neural audio codecs, and why are they fundamental to models such as VALL-E, xTTS, and CosyVoice?*
- *Compare spectrogram-based TTS models (e.g., Tacotron2) with codec-based language models (e.g., VALL-E). What are the advantages and disadvantages of each paradigm?*
- *Explain the concept of **zero-shot voice cloning**. What information is typically required to synthesize speech in a previously unseen speaker's voice?*
- *What are speaker embeddings? Why are they important for multi-speaker and voice-cloning TTS systems?*
- *Explain how **xTTS v2** enables multilingual speech synthesis while preserving speaker identity.*
- *What advantages do instruction-based TTS systems such as **Parler-TTS** provide compared to traditional speaker-conditioned models?*
- *What types of textual prompts can be used to control speech generation in instruction-following TTS models?*
- *Explain the difference between conditioning on **speaker identity**, **reference audio**, and **natural language descriptions**.*

- *Compare autoregressive, non-autoregressive, and diffusion/flow-based TTS architectures in terms of synthesis quality, inference speed, and robustness.*
- *Explain the intuition behind **Diffusion Models** for speech synthesis. Why is denoising performed iteratively?*
- *What is **Flow Matching**? How does it differ conceptually from diffusion-based generation?*
- *Why can Flow Matching achieve faster inference than traditional diffusion models?*
- *Explain the main idea behind **F5-TTS**. Why are flow-based methods becoming increasingly popular for speech synthesis?*
- *What objective metrics are commonly used to evaluate TTS systems? Why is objective evaluation alone often insufficient?*
- *How can human evaluation be performed for TTS systems? Explain metrics such as **Mean Opinion Score (MOS)**, **Similarity MOS (SMOS)**, and **Comparative MOS (CMOS)**.*
- *Why is speaker-independent evaluation important in multi-speaker TTS? What forms of data leakage can artificially inflate performance?*
- *Explain how pretrained speech representations, neural codecs, and large language models have changed the design of modern TTS systems compared to Tacotron2-era architectures.*
- *Why are LLM-generated edge-case texts useful when evaluating TTS systems? Give examples of challenging inputs such as abbreviations, numbers, code-switching, tongue twisters, rare words, and punctuation-heavy sentences.*
- *Compare **Tacotron2**, **Tortoise TTS**, **VALL-E**, **xTTS v2**, **CosyVoice 2**, **Parler-TTS**, and **F5-TTS** in terms of architecture, conditioning mechanism, generation strategy, and practical use cases.*
- *What are the current limitations of modern TTS systems, and what research directions are likely to shape the next generation of speech synthesis models?*
-

Appendix 2 (Practical Exercises)

Exercise 1: Sampling Strategies in GPT-based TTS

During inference, a GPT-based TTS model predicts the next acoustic token from a vocabulary. At one decoding step, the model outputs the following logits:

	A	B	C	D
Logit	2	1	1	0

The probabilities are computed using the softmax function:

$$P(y_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

For simplicity, use the following approximations:

$$e^2 \approx 8, \qquad e^1 \approx 3, \qquad e^0 = 1.$$

For the sampling-based decoding strategies, assume that the generated random number is

$$r = 0.75.$$

A token is sampled according to the cumulative probability distribution.

Tasks

1. Compute the softmax probabilities and select the next token using **greedy decoding**.
2. Apply **temperature scaling** with temperature $\tau = 2$:

   $$z_i' = \frac{z_i}{\tau}.$$

   Compute the new softmax probabilities and determine which token is sampled using $r = 0.75$.
3. Apply **top-k sampling** with $k = 2$. Compute the renormalized probabilities of the remaining tokens and determine which token is sampled using $r = 0.75$.
4. Apply **top-p (nucleus) sampling** with $p = 0.75$. Determine the smallest set of tokens whose cumulative probability is at least 0.75, compute the renormalized probabilities, and determine which token is sampled using $r = 0.75$.

Exercise 2: Self-Attention with Positional Encoding

Consider a sequence of three token embeddings:

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

Assume a simplified self-attention layer where

$$Q = K = V = X,$$

and attention is computed as

$$\text{Attention}(Q, K, V) = \text{Softmax}(QK^\top)V.$$

For simplicity, use:

$$e^0 = 1, \qquad e^1 \approx 3, \qquad e^2 \approx 8.$$

Now add sinusoidal-style positional encodings:

$$P = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}, \qquad X' = X + P.$$

Again assume

$$Q = K = V = X'.$$

Tasks

1. Compute the attention score matrix $QK^\top$ before adding positional encoding.
2. Compute the attention weights using row-wise softmax.
3. Compute the output embeddings after self-attention.
4. Add positional encoding and compute X'.
5. Repeat Steps 1–3 for X'.
6. Compare the attention weights and output embeddings with and without positional encoding. Explain why positional information changes the self-attention result.

Exercise 3: RVQ-VAE Inference with Two Codebook Layers

In modern codec-based TTS systems, speech can be represented using discrete acoustic tokens obtained from a neural codec. One common approach is Residual Vector Quantization (RVQ), where each layer quantizes the remaining residual from the previous layer.

Consider a sequence of three encoder outputs:

$$Z = \begin{bmatrix} 3 & 1 \\ 0 & 4 \\ 2 & 3 \end{bmatrix}.$$

RVQ consists of two quantization layers.

The first codebook is:

$$C^{(1)} = \begin{array}{c|cc} & x_1 & x_2 \\ \hline A & 2 & 0 \\ B & 0 & 2 \end{array}$$

The second codebook is:

$$C^{(2)} = \begin{array}{c|cc} & x_1 & x_2 \\ \hline C & 1 & 1 \\ D & 0 & 2 \end{array}$$

For each input vector z_t, RVQ inference is performed as follows:

$$q_t^{(1)} = \arg\min_{c \in C^{(1)}} \|z_t - c\|_2^2,$$

$$r_t = z_t - q_t^{(1)},$$

$$q_t^{(2)} = \arg\min_{c \in C^{(2)}} \|r_t - c\|_2^2,$$

$$\hat{z}_t = q_t^{(1)} + q_t^{(2)}.$$

Tasks

1. For each of the three input vectors, find the nearest codebook vector from the first RVQ layer.
2. Compute the residual vectors after the first RVQ layer.
3. For each residual vector, find the nearest codebook vector from the second RVQ layer.
4. Compute the final reconstructed vectors $\hat{z}_t$.
5. Write the resulting sequence of discrete acoustic tokens from both RVQ layers.